

Finite State Model — pqc_crypto_module v0.1.0

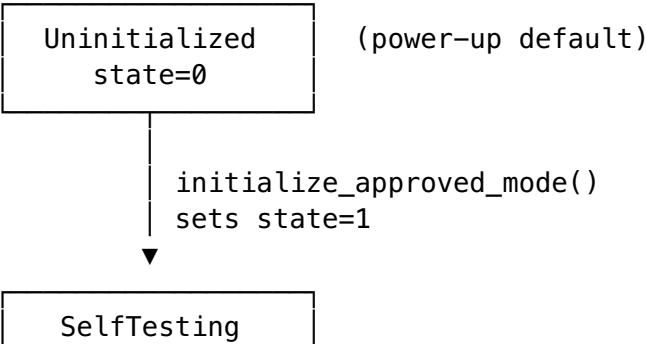
Disclaimer: This module is prepared for FIPS 140-3 evaluation and is not currently validated.

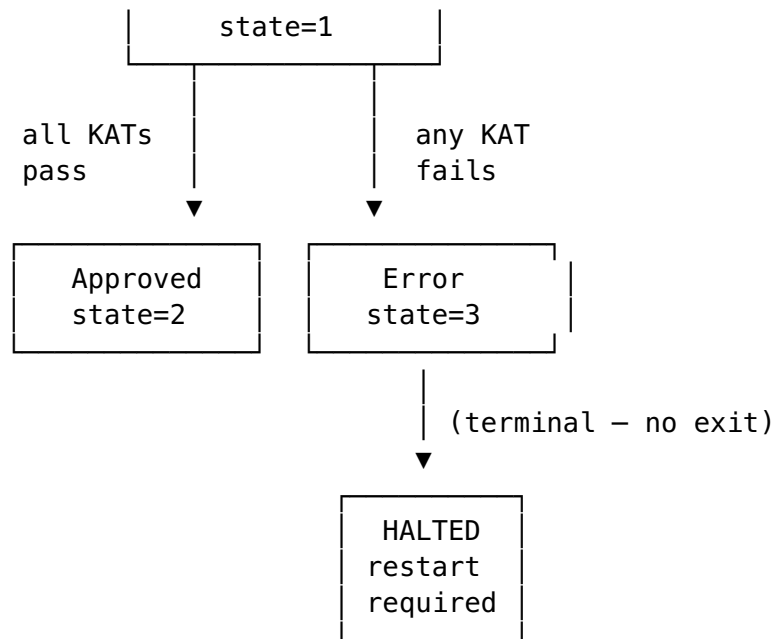
1. States

The module has four states, encoded as a u8 in a global AtomicU8 with SeqCst ordering:

State	Value	Description
Uninitialized	0	Power-up state. No cryptographic services available.
SelfTesting	1	Transient state during KAT execution. No services available.
Approved	2	Operational state. All approved services available.
Error	3	Terminal state. All services permanently rejected.

2. State Diagram





3. Transitions

From	To	Trigger	Condition
Uninitialized	SelfTesting	initialize_approved_mode() called	Always
SelfTesting	Approved	self_tests::run_all() returns Ok(())	All 4 KATs pass
SelfTesting	Error	self_tests::run_all() returns Err(...)	Any KAT fails

4. Forbidden Transitions

The following transitions are never performed by the module:

From	To	Reason
Error	Uninitialized	Error state is terminal
Error	SelfTesting	Error state is terminal
Error	Approved	Error state is terminal
Approved	Uninitialized	No downgrade path in production
Approved	SelfTesting	Re-initialization is not supported
Approved	Error	Self-tests only run once during initialization
Uninitialized	Approved	Must pass through SelfTesting
Uninitialized	Error	

From	To	Reason
		Must pass through SelfTesting

Note: A `__test_reset()` function exists to reset state to Uninitialized for integration tests. It is not part of the production API and is gated behind `#[doc(hidden)]`.

5. Fail-Closed Behavior

The module is designed to fail closed in all non-operational states:

Uninitialized state

All approved API calls (sign, verify, hash, encapsulate, decapsulate, random) return:

```
Err(CryptoError::ModuleNotInitialized)
```

SelfTesting state

The SelfTesting state is transient (exists only during the synchronous execution of `initialize_approved_mode()`). If another thread calls an API function during this window, it receives:

```
Err(CryptoError::ModuleNotInitialized)
```

Error state

All API calls return:

```
Err(CryptoError::ModuleInErrorState)
```

The module cannot recover from Error state. The process must be restarted.

Unknown state values

The `From` implementation for `ModuleState` maps any unrecognized value to `Error`, ensuring that memory corruption or unexpected state values result in fail-closed behavior.

6. Guard Functions

Guard	Location	Passes when	Used by
<code>require_approved()</code>	<code>approved_mode.rs</code>	<code>state == Approved</code>	All approved API functions
<code>ensure_not_approved()</code>	<code>legacy.rs</code>		

Guard	Location	Passes when	Used by
		state != Approved	All legacy (non- approved) functions

These guards are the first operation in every public API function. No cryptographic computation occurs before the guard check passes.

7. Concurrency

The state is stored in an AtomicU8 with SeqCst ordering, which provides:

- **Visibility:** State changes are immediately visible to all threads.
- **Atomicity:** State transitions cannot be partially observed.
- **No locks:** The state machine does not use mutexes, avoiding deadlock risk.

The `initialize_approved_mode()` function is intended to be called once at process startup. If called concurrently by multiple threads, each call will execute self-tests, and the last write wins. This is safe because:

- If all self-tests pass, the final state is `Approved`.
- If any self-test fails, the final state is `Error` (which is correct).
- The swap operation ensures no intermediate state is lost.

8. Test Coverage

Test file	What it verifies
tests/api_boundary.rs	All operations fail before init; all work after init
tests/approved_vs_legacy.rs	Legacy blocked in Approved; legacy works before init
tests/no_fallback.rs	No classical fallback on failure
tests/self_tests.rs	Self-tests run and pass